

JD Robot Cortex

Project Documentation

Perception, reasoning, memory, and safe action execution
for an EZ-Robot JD Humanoid on Synthiam ARC

JD Robot Cortex — Project Documentation

Non-technical overview

Architecture overview

Data flow

Repository layout

Running the system (`run.py`)

Main program (`jd_robot_system/main.py`)

Action execution and speech

Gemini integration (`jd_robot_system/gemini_brain.py`)

Vision pipeline (`vision_pipeline/01_full_pipeline.py`)

Current scene context (`jd_robot_system/scene_context.py`)

Persistent memory (`memory/`)

Auxiliary watcher processes

Setup and configuration

Data and privacy

What this documentation emphasizes

Troubleshooting

Notes

JD Robot Cortex — Project Documentation

This repository implements a helper system for an EZ-Robot JD humanoid running on Synthiam ARC. ARC owns the servos, speech synthesis, and the onboard camera; JD Cortex is a set of Python processes (plus one small C# ARC plugin) that run beside ARC and add everything ARC doesn't do on its own: safe command execution, natural-language understanding, vision-based scene awareness, persistent memory of who and what JD has seen, and hand-gesture control.

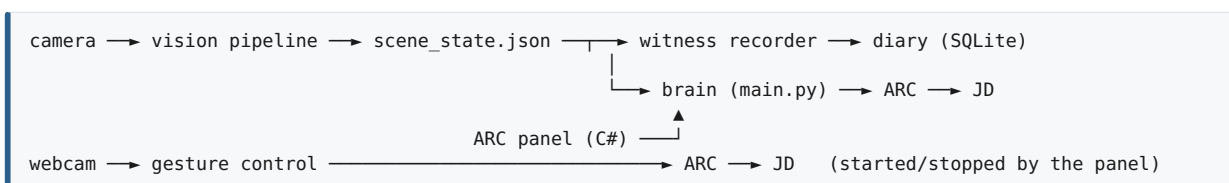
Non-technical overview

Think of this project as a set of helpers that let your JD robot:

- hear commands from you, either typed, spoken into a microphone, or via hand gestures in front of a webcam,
- think about what you said using Google Gemini, and reply out loud,
- move its body, play sounds, or light up — but only in ways that have been pre-approved as safe,
- watch the scene with a camera and notice who is there, what they're doing, and what they're holding,
- remember what it has seen, so it can later answer “who came by today?” or “what was I doing this morning?”,
- alert you if something unusual happens, like an unrecognised person or a sharp object in view,
- occasionally make a short unprompted remark when something in the scene changes, the way a companion in the room might.

The code is split into independent pieces, each with one job — sensing, remembering, deciding, and speaking are never done by the same process. That separation makes the system easier to reason about and safer to run.

Architecture overview



The system is intentionally decoupled into several cooperating processes, launched and supervised by `run.py`:

- `vision_pipeline/01_full_pipeline.py` — the sensor process: reads camera frames from ARC, detects people and objects, and writes structured scene state.
- `memory/witness_recorder.py` — reads that scene state and turns raw per-frame sightings into a durable, timestamped diary of arrivals, departures, and notable objects.
- `jd_robot_system/main.py` — the brain: takes typed, spoken, or panel input, talks to Gemini, validates and executes actions, and owns the one ARC connection that speaks.
- `jd_robot_system/surveillance_watcher.py` — offline scene monitoring for safety alerts (unknown people, sensitive objects).
- `jd_robot_system/ambient_watcher.py` — optional, Gemini-driven unprompted remarks about scene changes.
- `memory/gesture_control.py` — a separate process, started on demand from the ARC panel, that reads a stationary webcam and drives JD's walking/pose actions from hand shapes.
- `memory/plugin/` — a C# ARC Behavior Control skill (“Memory”) that gives ARC a live panel: connection status, a scene card, example questions, one input box for both memory questions and

commands, hold-to-talk, and a gesture toggle.

A typical deployment uses `python run.py` to start the Python processes in the order they depend on each other, while ARC (with the Memory skill loaded) remains the single central robot controller.

Data flow

1. The vision pipeline requests camera snapshots from ARC and converts them into structured scene state, written to `vision_pipeline/scene_state.json`.
2. `memory/witness_recorder.py` polls that file, debounces sightings into confirmed arrivals/departures/held-objects, and records diary events into a SQLite database (`memory/witness_diary.sqlite3`).
3. `jd_robot_system/main.py` reads the *current* scene through `scene_context.py` and the *past* diary through `memory_context.py` on every command — these stay as two separate prompt blocks so Gemini can't confuse "seeing now" with "saw earlier".
4. User commands arrive by typing, by local microphone voice input, or through the ARC panel's text box or hold-to-talk button (which all funnel through the same `process_command()` path).
5. `main.py` first tries a local literal match against the known safe action list. If none matches, the command (plus scene summary, diary block, and recent conversation history) is sent to `gemini_brain.understand()`.
6. Gemini returns a short spoken reply and, optionally, a suggested action.
7. `main.py` speaks the reply through ARC via `tts.py`.
8. If an action was suggested, it is validated again against `known_actions.py` before ever reaching ARC — Gemini is never trusted to drive the robot directly.
9. Separate watcher processes (`surveillance_watcher.py`, `ambient_watcher.py`, and the witness recorder in announce mode) also read `scene_state.json` and queue speech through the file-backed `speech_queue.py`; only `main.py` ever opens an ARC connection for speaking.
10. Hand gestures, when enabled from the panel, are read from a separate stationary webcam and sent to ARC over their own TCP connection — independent of the brain's connection, so a gesture-control crash can't take conversation down with it.

This separation keeps sensing, remembering, deciding, and acting distinct, which simplifies safety reasoning and avoids cross-process contention on the single ARC connection.

Repository layout

- `run.py` — single launcher and pre-flight checker for the whole system (see below). Replaces the old `start_jd.bat`.
- `jd_robot_system/` — the brain: main control loop, Gemini integration, ARC connection, TTS, local and panel voice input, scene/memory context bridging, known-action whitelist, and the panel's TCP server.
- `vision_pipeline/` — camera snapshot acquisition, YOLO object/pose detection, face recognition, and scene state output.
- `memory/` — persistent witness diary (SQLite store + recorder), hand-gesture control, and the C# ARC panel plugin (`memory/plugin/`).
- `setup/` — GPU check and face enrollment helpers.
- `shared/` — a standalone, older `jd_command_and_act.py` command-and-act script plus its own copies of `known_actions.py` and `servo_map.py`, kept for reference/experimentation outside the main pipeline.
- `voice_model/parakeet/` — placeholder for the local Parakeet speech-to-text model; only `tokens.txt` is committed, the `.onnx` weight files must be supplied separately.
- `requirements.txt` — Python dependencies for vision, speech, gestures, and Gemini integration.

Running the system (`run.py`)

`run.py` finds the repo from its own location (so it works from any folder on any machine) and starts the Python processes in the order they actually depend on each other.

```
python run.py check      # what's ready, what's missing – run this first
python run.py            # start everything
python run.py --no-ambient # everything except ambient mode (saves Gemini quota)
python run.py --minimal   # ARC brain + witness memory only, no vision
```

`check` reports, in one place:

- whether every required library imports (`cv2` , `mediapipe` , `requests` , `sounddevice` , `sherpa_onnx` , `ultralytics` , `face_recognition`),
- whether `setup/known_encodings.pkl` and the Parakeet `.onnx` weights exist,
- whether the YOLO weight files (`yolov8m.pt` , `yolov8m-pose.pt`) are present or will download on first run,
- how many Gemini API keys are set in the environment,
- whether ARC is listening on port 6666 and whether the panel port 5005 is free,
- whether any stale files are left over from a previous run (these are removed automatically before a normal start).

`start` launches, in order: the vision pipeline (waiting ~8 seconds for its YOLO models to load before starting anything that reads its output), the witness recorder, the brain (`main.py`), and — unless `--minimal` is passed — the surveillance watcher and (unless `--no-ambient` is passed) the ambient watcher. Each process gets its own console window on Windows. Ctrl+C in the launcher's own window stops all child processes cleanly.

Main program (`jd_robot_system/main.py`)

Role

`main.py` is the orchestrator that:

- connects to ARC over TCP using `arc_connection.py` ,
- starts the panel's TCP server (`panel_server.py`) so the ARC plugin can talk to the same brain,
- chooses the console input mode (typed or local voice),
- processes every command from whichever source it came from,
- speaks replies,
- executes safe robot actions,
- handles queued speech from auxiliary watchers.

ARC connection

`arc_connection.py` maintains a single TCP socket to ARC. All ARC commands are sent through this one connection, and it is opened only once per `main.py` run — this is the one process in the whole system that ever calls ARC's `SayWait` .

Input modes

- **Typed mode:** the user types commands in the console.
- **Voice mode (console):** `voice_parakeet.listen_for_command()` waits for Enter, records until silence, and transcribes with a local Parakeet model.
- **Panel free text and hold-to-talk:** the ARC plugin sends free text or a hold-to-talk recording to `panel_server.py` (port 5005), which routes it through the exact same `process_command()` path, serialised with the console input by a shared lock so replies never interleave on the one ARC connection.

Command processing flow

For each input:

1. `main.py` checks for forget/reset phrases and clears short-term conversation memory if requested.
2. It tries a local literal match against known movements, sounds, and lights.
3. If a local match succeeds, the action is validated and executed immediately.
4. If no local match is found, the command is sent to `gemini_brain.understand()`.
5. `understand()` builds a prompt that includes:
 - the user's text,
 - the current scene summary from `scene_context.py` (if fresh vision data exists),
 - the witness diary block from `memory_context.py` (if any events have been recorded),
 - recent conversation history from `conversation_memory.py`,
 - the known safe actions from `known_actions.describe_all_known()`.
6. Gemini returns a short spoken reply and an optional matched action, in a strict two-line format the code parses.
7. `main.py` speaks the reply via `tts.speak()`.
8. If an action is suggested, it is validated again against `known_actions.py` before executing — Gemini's suggestion is never trusted on its own.

Safety and validation

The system uses a whitelist of permissible actions in `jd_robot_system/known_actions.py`:

- `MOVEMENTS` — pre-approved Auto Position actions.
- `SOUNDS` — valid Soundboard v4 track numbers.
- `LIGHTS` — valid RGB Animator effects.

Gemini may suggest an action, but the system never executes it without checking that it matches one of those known entries exactly.

Action execution and speech

Robot actions

`known_actions.py` translates safe commands into ARC `ControlCommand(...)` calls:

- movements use `ControlCommand("Auto Position", AutoPositionAction, "<title>")`,
- sounds use `ControlCommand("Soundboard v4", Track_<n>)`,
- lights use `ControlCommand("RGB Animator", AutoPositionAction, "<title>")`.

After a movement, `main.py` returns the robot to the `Standing` pose (via `AutoPositionFrameJump`) so it isn't left in an awkward position.

Speech output

`jd_robot_system/tts.py` sends `SayWait(...)` via ARC and blocks until ARC acknowledges. It also sets a `speaking_flag` so local microphone recording (console or panel) doesn't begin while JD is still talking.

Cross-process speech queue

`jd_robot_system/speech_queue.py` implements a simple file-backed queue (`speech_queue.json`).

- Watcher processes (surveillance, ambient, and the witness recorder when `JD_ANNOUNCE=on`) append messages using `request_speech()`.
- `main.py` polls `speech_queue.pop_pending_messages()` on a background thread and speaks queued messages through the same single ARC connection.

This design ensures only one process ever speaks through ARC, preventing conflicts.

Gemini integration (`jd_robot_system/gemini_brain.py`)

Purpose

Gemini is used for conversational understanding, matching free-form requests to real robot actions, judging whether a surveillance condition is worth alerting on, and deciding whether an ambient scene change is worth a remark. It is never used to drive the robot directly without verification against the known-safe list.

Request pipeline and quota resilience

- Gemini calls are made via plain HTTP requests to Google's `generateContent` endpoint — no SDK required.
- The code walks `GEMINI_MODEL_CANDIDATES` in order; if a model returns HTTP 404 (deprecated/unavailable), the next model is tried.
- For each model, every configured API key (`GEMINI_API_KEY` , and optionally `GEMINI_API_KEY_2` / `GEMINI_API_KEY_3`) is tried in turn — the free tier allows roughly 20 requests per day per key per model, so multiple keys give the system extra pools to rotate through in one session.
- Network errors and timeouts are retried a configurable number of times (`GEMINI_MAX_RETRIES` , `GEMINI_RETRY_DELAY`) before moving on.
- A running usage tally (`usage_summary()`) is exposed to the ARC panel so quota drain is visible live during a session, instead of only being discovered afterward as unexplained silence.

Prompt formats

Each Gemini use asks for a strict, parseable response format:

- `understand()` — conversation and action matching: `REPLY: <spoken reply>` and `MATCH: <category>|<name>` , where `category` is exactly `movement` , `sound` , or `light` , or `MATCH: NONE` .
- `assess_alert()` — used by the surveillance watcher: `ALERT: <yes|no>` and `LINE: <spoken alert or NONE>` . If Gemini is unavailable, this fails safe by alerting anyway with a generic line, rather than staying silent on a possible real event.
- `ambient_comment()` — used by the ambient watcher: `COMMENT: <remark or NONE>` . This one fails silently (returns `None`) if Gemini is unavailable, since ambient remarks are not safety-critical.

The strict two/one-line formats keep parsing simple and reliable across all three uses.

Vision pipeline (`vision_pipeline/01_full_pipeline.py`)

Role

The vision pipeline is a pure sensor process:

- it connects to ARC via `vision_pipeline/arc_client.py` ,
- requests camera snapshots using ARC's `Camera Snapshot` skill,
- runs object detection, pose tracking, and face recognition,
- derives simple scene state,
- writes it to `vision_pipeline/scene_state.json` .

It does not speak, execute robot actions, or depend on Gemini for its main output.

Visual analysis

The pipeline combines:

- YOLO object detection for general objects,
- YOLO pose tracking for people and their track IDs,
- face recognition against `setup/known_encodings.pkl` to label known people,

- logic to infer whether a tracked person is holding a known object or sitting on furniture,
- posture estimation by comparing person bounding-box height over time.

Output format

`vision_pipeline/scene_state.json` contains:

- `timestamp` — when the scene state was produced,
- `objects_visible` — list of detected objects (excluding people),
- `people` — a dictionary keyed by tracker ID, each with `name`, `posture`, `holding`, `sitting_on`.

The file is written atomically (temporary `.tmp` file, then `os.replace(...)`) so readers never see a half-written file.

Snapshot requests

`surveillance_watcher.py` can request a saved picture by writing `vision_pipeline/snapshot_request.txt`.

When the vision pipeline notices that file, it writes the current annotated frame into

`jd_robot_system/snapshots/` and deletes the request.

Current scene context (`jd_robot_system/scene_context.py`)

`scene_context.py` reads `vision_pipeline/scene_state.json` fresh on every call and converts it into a short, plain-English summary for the Gemini prompt.

- If the file is missing, unreadable, or older than 10 seconds, it returns `None` — the prompt simply omits the vision section rather than working from stale data.
- If no people are visible, it returns `"No one is currently visible."`
- Otherwise it describes each visible person by name (or “someone JD doesn’t recognise” for unnamed tracks — never a raw tracker ID, since the same person can churn through many IDs), plus posture, held object, and furniture, when known.

This is deliberately about the *present* only; anything about the past comes from the witness diary instead.

Persistent memory (`memory/`)

This is what lets JD answer questions like “who came by this morning?” using the same model and prompt that handles ordinary conversation — there is no separate memory-answering path.

Witness recorder (`memory/witness_recorder.py`)

Runs alongside the vision pipeline. It polls `scene_state.json` once a second and turns raw per-frame sightings into a small number of meaningful diary events:

- **Identity is tracked by name, never by tracker ID.** The same person can burn through dozens of tracker IDs in a minute as tracking drops and re-acquires, so every unrecognised sighting pools into one `"unknown"` bucket, and each named person is one entry.
- An **arrival** needs a couple of seconds of continuous presence before it’s logged (`ARRIVE_CONFIRM_SECS`); a **departure** needs a longer solid gap of absence (`LEAVE_CONFIRM_SECS`) before it’s logged, with the departure’s duration measured back to the last real sighting. This debouncing is what keeps the diary readable instead of flapping on every tracking hiccup.
- Holding/sitting events are only logged for named, confirmed-present people, and the same object is only re-logged after a quiet gap (`RELOG_GAP_SECS`), so a flickering detection doesn’t fill the diary with the same object twenty times.
- Ambient objects (`objects_visible`) get a lighter version of the same debounce: one “came into view” event per continuous appearance, with no departure event — “the chair left” isn’t useful information.

- With `JD_ANNOUNCE=on` in the environment, confirmed arrivals are also dropped into `speech_queue.json` so `main.py`'s alert thread speaks them — this process never opens its own ARC connection.
- Old events are pruned at startup, keeping `JD_DIARY_KEEP_DAYS` (default 14) days of history.

Run it standalone: `python memory/witness_recorder.py`. Logic self-test with no camera: `python memory/witness_recorder.py selftest`.

Witness store (`memory/witness_store.py`)

A small SQLite layer (`memory/witness_diary.sqlite3`, WAL mode) that both the recorder (writer) and `memory_context.py` (reader) use, each opening a short-lived connection per operation so neither process can block or hold the database hostage if it crashes.

`diary_context()` builds the compact plain-text block that goes into the Gemini prompt: who was seen today (with time spans), who was last seen on earlier days, and the most recent individual diary entries — trimmed by whole entries (oldest first) to stay under a character budget, rather than cutting a line mid-sentence.

Memory bridge (`jd_robot_system/memory_context.py`)

Bridges the witness diary into the Gemini prompt, exactly like `scene_context.py` does for vision: optional, read fresh per command, and an empty or missing diary just means the prompt goes out without a memory section. Current scene and past sightings stay in separate prompt blocks so the model can't confuse "seeing now" with "saw earlier".

Hand-gesture control (`memory/gesture_control.py`)

A separate process, started and stopped from the ARC panel's gesture toggle (or run standalone). It classifies a hand shape from a **stationary laptop/USB webcam** — deliberately never JD's own camera, since JD's camera moves when JD does, which would shake the view, lose the hand, and trip the safety stop the moment a FORWARD gesture worked.

Gestures and their actions:

Gesture	Action
fist	forward
open hand	stop
index pointing left / right	turn left / right
index pointing down	sit
peace sign (index + middle)	wave
three fingers (index + middle + ring)	stand
index + pinky	pushups

A gesture must hold stable for several consecutive frames (`STABLE_FRAMES`) before it's committed and sent to ARC over its own TCP connection — independent of the brain's connection. If the hand disappears while JD is walking or turning, a safety stop is sent automatically after a short number of hand-less frames (`NO_HAND_STOP_FRAMES`), so a lost hand can never leave JD walking indefinitely.

Run standalone: `python memory/gesture_control.py`. Logic self-test with no camera or MediaPipe: `python memory/gesture_control.py selftest`.

ARC panel plugin (`memory/plugin/`)

A C# ARC Behavior Control skill ("Memory") built with Visual Studio from files generated by ARC's Create Robot Skill wizard. It gives ARC a live panel showing:

- a connection status light,
- a live “what JD sees” card,
- clickable example questions,
- one input box for both memory questions and free-text commands,
- a hold-to-talk button,
- a hand-gesture-control toggle with a plain-language legend,
- a guarded shutdown button.

It talks to `panel_server.py` over `127.0.0.1:5005` using a simple line protocol (`status`, `diary [n]`, `alerts [n]`, `gesture on|off`, `listen start|stop`, `stop`, or free text for the brain). It also captures JD’s own camera frames and writes them to a shared bridge folder for the Python side, asking the running Python process for that folder’s path on connection rather than hardcoding it.

Building requires Windows and a working ARC + SDK install: open `MY_PROJECT_NAME.csproj`, build, and the DLL is placed into ARC’s plugin folder automatically; then in ARC, **Project → Add Skill → Beta → Memory**.

Auxiliary watcher processes

Surveillance watcher

`jd_robot_system/surveillance_watcher.py` monitors `vision_pipeline/scene_state.json` for unknown people and sensitive objects (such as knives or scissors). A condition must persist across multiple polls before it triggers, and alerting is rate-limited to avoid repeated alarms. When an alert is warranted, it:

- logs the event to `jd_robot_system/activity_log.txt`,
- requests a camera snapshot,
- queues a speech alert through `speech_queue.py`,
- optionally asks `gemini_brain.assess_alert()` to judge severity and word the alert appropriately, failing safe (alert anyway, generic wording) if Gemini is unavailable.

This watcher’s core detection logic is fully offline; only the alert-wording step touches the network.

Ambient watcher

`jd_robot_system/ambient_watcher.py` monitors the same scene state for meaningful changes — arrivals, departures, posture changes, long idle states — and asks `gemini_brain.ambient_comment()` whether a short, companion-style remark is appropriate right now. If Gemini produces a comment, it’s queued through `speech_queue.py`.

Unlike the surveillance watcher, this component depends on Gemini/network availability and skips silently when Gemini is unavailable, since ambient remarks are a non-critical enhancement, not a safety feature.

Setup and configuration

`jd_robot_system/config.py`

Key settings:

- `ARC_HOST` / `ARC_PORT` — where ARC’s TCP script server is listening (default `127.0.0.1:6666`).
- `GEMINI_API_KEY`, `GEMINI_API_KEY_2`, `GEMINI_API_KEY_3` — read from the environment first, falling back to the value in `config.py`. Any that are set, non-empty, and not the placeholder are collected into `GEMINI_API_KEYS` for rotation.
- `GEMINI_MODEL_CANDIDATES` — fallback model names, tried in order.
- `GEMINI_TIMEOUT`, `GEMINI_MAX_RETRIES`, `GEMINI_RETRY_DELAY` — per-call network resilience settings.
- `SPEECH_POLL_INTERVAL` — only relevant if using ARC’s own Speech Recognition skill instead of local

Parakeet voice input.

Face enrollment

`setup/enroll_faces.py` generates `setup/known_encodings.pkl` from photos placed in `setup/known_faces/`. The vision pipeline uses that file to label known people in the camera feed. Photos and encodings are gitignored and should stay that way — this repository holds source code, not anyone’s photograph.

Environment variables used across the system

Variable	Purpose	Default
<code>GEMINI_API_KEY</code> / <code>_2</code> / <code>_3</code>	Gemini API keys to rotate through	placeholder (invalid)
<code>JD_ANNOUNCE</code>	witness recorder speaks arrivals when <code>on</code>	<code>off</code>
<code>JD_DIARY_KEEP_DAYS</code>	how many days of diary history to retain	<code>14</code>
<code>JD_DIARY_DB</code>	override the diary’s SQLite file path	<code>memory/witness_diary.sqlite3</code>
<code>JD_SCENE_STATE</code>	override the scene-state file path	<code>vision_pipeline/scene_state.json</code>
<code>JD_GESTURE_CAMERA</code>	webcam index for gesture control	<code>0</code>
<code>JD_ARC_HOST</code> / <code>JD_ARC_PORT</code>	ARC address, as seen by gesture control	<code>127.0.0.1</code> / <code>6666</code>
<code>JD_GESTURE_WINDOW</code>	show the live annotated gesture window	<code>on</code>
<code>JD_ACTIVITY_LOG</code>	override the surveillance activity log path	<code>jd_robot_system/activity_log.txt</code>
<code>JD_BRIDGE_DIR</code>	fallback bridge folder for older plugin builds	<code><repo>/bridge</code>

Dependencies

`requirements.txt` lists the Python packages required for the vision pipeline, face recognition, Gemini integration, local speech recognition, and hand-gesture control. Notably, `mediapipe` is pinned to `0.10.21`, since later releases removed the legacy Hands API gesture control relies on. GPU (CUDA) support for the vision pipeline requires installing a matching PyTorch build separately before the rest of `requirements.txt`.

The repository also relies on external assets not tracked in git:

- YOLO object and pose model weights (`yolov8m.pt`, `yolov8m-pose.pt`), downloaded automatically on first run,
- Parakeet `.onnx` files for local voice recognition,
- enrolled face photos and their encodings,
- a valid Gemini API key for any conversational or Gemini-driven feature.

Data and privacy

The witness diary keeps `JD_DIARY_KEEP_DAYS` (default 14) days of events and prunes older ones automatically at every recorder startup.

Four things are deliberately excluded from version control: the diary database, enrolled face photos and their encodings, captured snapshots, and downloaded model weights. This repository holds source code, not runtime data and not anyone’s photograph.

What this documentation emphasizes

- **Clear data boundaries:** the vision pipeline only writes sensor state; the witness recorder only turns that into a durable diary; the main program only speaks and acts.
- **Single ARC speaker owner:** only `main.py` ever calls ARC's `SayWait`; gesture control uses an entirely separate ARC connection for movement commands only.
- **Safe action whitelisting:** Gemini is used for understanding, but every robot action is validated against a known-safe list before it ever reaches ARC.
- **Identity by name, not tracker ID:** both the live scene summary and the witness diary describe people by recognised name (or a pooled "unknown"), never by a raw, unstable tracker ID.
- **Fail-safe vs. fail-silent, deliberately different per feature:** a surveillance alert defaults to alerting when Gemini is unavailable, since silence could hide a real event; an ambient remark defaults to saying nothing, since a skipped remark costs nothing.
- **Optional features degrade gracefully:** vision, memory, gestures, and Gemini are all optional enhancements layered on top of the core typed-command path, not hard dependencies of it.

Troubleshooting

Run `python run.py check` first — it answers most of the questions below directly.

Symptom	Likely cause
ARC won't connect	TCP Script Server skill not running on port 6666
Gemini fails every call	usually quota, not a bad key — watch the panel's usage counter
Nobody gets named	<code>known_encodings.pkl</code> missing — run <code>setup/enroll_faces.py</code>
Voice input does nothing	Parakeet <code>.onnx</code> files missing, or no microphone available
Panel connects, scene card stays empty	vision pipeline isn't running or isn't writing <code>scene_state.json</code>
Nothing is remembered	<code>memory/witness_recorder.py</code> isn't running
Gesture toggle fails to turn on	webcam already in use (often by ARC's own Camera skill pointed at the same device)

Self-tests that need no camera or ARC connection:

```
python memory\witness_recorder.py selftest
python memory\gesture_control.py selftest
```

To see which Gemini models a key can currently reach:

```
cd jd_robot_system
python -c "from gemini_brain import list_available_models; list_available_models()"
```

There is no automated end-to-end test suite; these self-tests are the substitute for the logic that can be tested without hardware.

Notes

- The repository does not include Synthiam ARC itself, nor the JD robot hardware.
- The `GEMINI_API_KEY` placeholder in `jd_robot_system/config.py` is not valid until replaced with a real key (or set via environment variables).
- The Parakeet `.onnx` voice model files are not included and must be obtained separately.

- `shared/jd_command_and_act.py` (plus its own `known_actions.py` / `servo_map.py`) is an older, standalone command-and-act script kept for reference; the actively developed path is `jd_robot_system/` + `run.py`.
- This project has been developed and tested on Windows; ARC and its plugin SDK are Windows-only, so the system as a whole has not been fully tested on Linux or macOS.